14. Keylime 테스트 전략 및 품질 보증 분석

14.1 개요

Keylime 프로젝트는 TPM 기반 원격 증명 시스템의 복잡성과 보안 중요성을 고려하여 포괄적인 테스트 전략과 품질 보증 체계를 구축하고 있습니다. 이 문서는 Keylime의 테스트 구조, 품질 관리 도구, CI/CD 파이프라인, 그리고 성능 검증 방법론을 상세히 분석합니다.

14.2 테스트 디렉토리 구조 분석

14.2.1 기본 테스트 구조

```
test/
├── run_tests.sh # 메인 테스트 실행 스크립트
├── test-requirements.txt # 테스트 전용 의존성
├── utils.py # 테스트 유틸리티 함수
├── __init__.py # 테스트 패키지 초기화
├── data/ # 테스트 데이터 및 설정
| ├── ca/ # 인증서 관련 테스트 데이터
| ├── config/ # 설정 파일 테스트 데이터
| ├── ima_keys/ # IMA 서명 검증용 키
| ├── templates/ # 설정 템플릿 테스트
| └── runtime-policy-*.json # 런타임 정책 테스트 파일
```

```
└── test_*.py # 개별 테스트 모듈들
```

14.2.2 테스트 실행 환경 설정

`test/run_tests.sh` 스크립트는 다음과 같은 포괄적인 테스트 환경을 구성합니다:

```
# 가상 환경 생성 및 활성화
python3 -m venv --system-site-packages /tmp/keylime_test
source /tmp/keylime_test/bin/activate

# 테스트 의존성 설치
pip3 install -r $KEYLIME_SRC/test/test-requirements.txt

# Keylime 설치 및 설정 생성
pip install . -r requirements.txt
python3 -m keylime.cmd.convert_config --defaults --out $KEYLIME_CONF_DIR
```

14.3 테스트 분류 및 구조

14.3.1 단위 테스트 (Unit Tests)

14.3.1.1 핵심 컴포넌트 테스트

TPM 관련 테스트:

- `test_crypto.py` : 암호화 기능 검증
- RSA/ECC 키 생성 및 암호화/복호화

- AES 대칭키 암호화
- HMAC 기반 메시지 인증
- 디지털 서명 생성 및 검증

```
# test_crypto.py 예시
def  test_rsa(self):
message = b"a secret message!"
private = rsa_generate(2048)
pubkeypem = rsa_export_pubkey(private)
ciphertext = rsa_encrypt(pubkey, message)
plain = rsa_decrypt(key, ciphertext)
self.assertEqual(plain, message)
```

설정 관리 테스트:

- `test_config.py` : 설정 파일 파싱 및 환경 변수 처리
- 다중 설정 파일 병합
- 환경 변수 우선순위 처리
- 설정 버전 호환성 검증

데이터베이스 테스트:

- `test_verifier_db.py` : SQLAlchemy ORM 및 데이터 무결성
- 에이전트 상태 관리
- 정책 연관 관계 검증
- 세션 관리 및 트랜잭션 처리

14.3.1.2 IMA 검증 테스트

test_ima_verification.py 는 파일 무결성 측정 아키텍처의 핵심 기능을 검증합니다:

```
# IMA 측정 목록 검증
def  test_measurment_verification(self):
measurements = MEASUREMENTS.splitlines()
_, failure = ima.process_measurement_list(
AgentAttestState("1"), measurements, RUNTIME_POLICY_TEST
)
self.assertTrue(not failure)


# 디지털 서명 검증
def  test_signature_verification(self):
keyrings = file_signatures.ImaKeyrings()
tenant_keyring = file_signatures.ImaKeyring()
# RSA 및 ECC 키를 사용한 서명 검증
```

14.3.2 통합 테스트 (Integration Tests)

14.3.2.1 컴포넌트 간 통신 테스트

통합 테스트는 다음 영역을 포괄합니다:

Agent-Verifier 통신:

- TPM 증명 프로세스 전체 흐름
- 상호 TLS 인증 검증
- 메시지 형식 및 프로토콜 준수성

Registrar-Agent 등록:

- EK/AK 키 교환 프로세스
- MakeCredential/ActivateCredential 동작
- 인증서 체인 검증

14.3.2.2 데이터베이스 통합 테스트

`test_verifier_db.py` 의 관계형 데이터 테스트:

```
def  test_11_relationship_access_after_session_commit(self):
"""세션 커밋 후 관계 접근 테스트 (DetachedInstanceError 수정)"""
with session_manager.session_context(self.engine) as session:
stored_agent = session.query(VerfierMain)
.options(joinedload(VerfierMain.ima_policy))
.filter_by(agent_id=agent_id).first()
# 세션 종료 후에도 관계 접근 가능 확인
self.assertIsNotNone(stored_agent.ima_policy)
```

14.3.3 보안 테스트 (Security Tests)

14.3.3.1 암호화 알고리즘 검증

TPM 암호화 테스트:

- TPM 2.0 표준 준수성 검증
- 하드웨어/소프트웨어 TPM 호환성
- 키 생성 및 관리 보안성

TLS 상호 인증 테스트:

- 인증서 검증 체인
- 암호화 스위트 협상
- 세션 보안 유지

14.3.3.2 IMA 보안 검증

파일 무결성 테스트:

- 런타임 정책 검증
- 디지털 서명 유효성
- 키링 관리 보안성

```
# 서명된 런타임 정책 검증
def  test_read_allowlist(self):
runtime_policy_envelope_raw, runtime_policy_key = ima.read_runtime_policy(
allowlist_file_signed, None, allowlist_dsse_key
)
# DSSE 서명 검증
ima.verify_runtime_policy(runtime_policy, runtime_policy_key)
```

14.4 TPM 에뮬레이터 기반 테스트 환경

14.4.1 SWTPM 에뮬레이터 설정

`.ci/test_wrapper.sh` 는 소프트웨어 TPM 에뮬레이터를 사용한 테스트 환경을 구성합니다:

```
# swtpm2 설정
```

```
mkdir  /tmp/tpmdir

swtpm_setup  --tpm2 \

--tpmstate /tmp/tpmdir \

--createek  --decryption  --create-ek-cert \

--create-platform-cert \

--display



# TPM 소켓 서버 시작

swtpm  socket  --tpm2 \

--tpmstate dir=/tmp/tpmdir \

--flags  startup-clear \

--ctrl type=tcp,port=2322 \

--server  type=tcp,port=2321 \

--daemon



# TPM2 ABRMD (Access Broker & Resource Manager Daemon) 시작

tpm2-abrmd \

--logger=stdout \

--tcti=swtpm: \

--flush-all \

--allow-root &
```

14.4.2 에뮬레이터 환경의 장점

일관된 테스트 환경:

- 하드웨어 TPM 없이도 전체 기능 테스트 가능
- CI/CD 환경에서 재현 가능한 테스트
- 다양한 TPM 상태 시뮬레이션

보안 테스트 지원:

- TPM 초기화 및 리셋 시나리오
- 키 생성 및 증명 프로세스 검증
- PCR 측정 및 확장 동작 테스트

14.5 CI/CD 파이프라인 및 자동화

14.5.1 GitHub Actions 워크플로우

14.5.1.1 메인 테스트 워크플로우 (`.github/workflows/test.yml`)

```yaml
jobs:
style-checks:
runs-on: ubuntu-latest
steps:
- uses: actions/setup-python@v5.3.0
- uses: pre-commit/action@v3.0.1

tpm-tests:
runs-on: ubuntu-latest
container:
```

```yaml
image: quay.io/keylime/keylime-ci:latest

options: --user root --mount type=tmpfs,destination=/var/lib/keylime/

steps:

- name: Run tests

run: .ci/test_wrapper.sh



lint:

runs-on: ubuntu-latest

steps:

- name: Run lints

run: tox -vv -e 'pylint,pylint310'

- name: Run mypy

run: tox -vv -e 'mypy'

- name: Run pyright

run: tox -vv -e 'pyright'
```

14.5.1.2 코드 커버리지 측정

커버리지 수집:

- `submit-PR-coverage.yaml` : Packit 테스트 결과 수집
- Codecov.io 통합으로 커버리지 리포팅
- 단위 테스트, 통합 테스트, E2E 테스트별 분리 측정

```
# 커버리지 플래그별 분리
```

```yaml
flags:
testsuite:
carryforward: false
unittests:
carryforward: false
packit-e2e:
carryforward: false
```

14.5.2 테스트 의존성 관리

14.5.2.1 테스트 전용 의존성 (`test/test-requirements.txt`)

```
coverage==4.5.2 # 코드 커버리지 측정
green==4.0.2 # Python 3.12+ 호환 테스트 러너
pytest-asyncio==0.10.0 # 비동기 테스트 지원
```

14.5.2.2 Tox 환경 관리 (`tox.ini`)

```ini
[tox]
envlist = py3,py310,pylint,pylint39

[testenv:pylint]
deps = pylint
commands = bash scripts/check_codestyle.sh
```

```
[testenv:mypy]
deps = mypy
commands = mypy keylime/


[testenv:pyright]
deps = pyright
commands = pyright
```

14.6 코드 품질 검사 도구

14.6.1 정적 분석 도구

14.6.1.1 Pylint 설정 (`pylintrc`)

```
[MASTER]
jobs=0
load-plugins=keylime.models.base.pylint


[MESSAGES CONTROL]
# 비활성화된 검사: 코드 스타일(C0103), 문서화(C0111) 등
disable=C0103,C0111,C0115,C0116,C0301,C0302,W0511,W0603


[TYPECHECK]
# 무시할 모듈: zmq, alembic, cryptography 등
```

```
ignored-modules=zmq,alembic.op,cryptography.hazmat.primitives
```

14.6.1.2 MyPy 타입 검사 (`mypy.ini`)

```
[mypy]
plugins = sqlmypy
strict = True
follow_imports = silent
ignore_missing_imports = True

# 파일별 세부 설정
[mypy-keylime.models.*]
disallow_untyped_defs = False
check_untyped_defs = False

[mypy-keylime.models.base.*]
disallow_untyped_defs = True
check_untyped_defs = True
```

14.6.2 코드 포맷팅 도구

14.6.2.1 Black 포맷터 (`pyproject.toml`)

```
[tool.black]
```

```
line-length = 120

target-version = ['py37']

extend-exclude = 'test/data/template-invalid-adjust/2.0/adjust.py'



[tool.isort]

profile = "black"

line_length = 120
```

14.6.2.2 코드 스타일 검사 스크립트

`scripts/check_codestyle.sh` 는 전체 프로젝트에 대해 Pylint 검사를 수행합니다:

```
${PYLINT} \

*.py \

$(find ./keylime ./test ./scripts -name '*.py' \

!  -name  'oldtest.py' \

! -path './keylime/da/examples/*.py')
```

14.7 성능 테스트 및 부하 테스트 방법론

14.7.1 현재 성능 테스트 현황

기존 성능 측정:

- 코드 커버리지를 통한 테스트 효율성 측정
- CI/CD 파이프라인 실행 시간 모니터링

- 메모리 사용량 및 리소스 최적화 검증

성능 테스트 부재 영역:

현재 Keylime 프로젝트에는 명시적인 성능 테스트나 부하 테스트 프레임워크가 구현되어 있지 않습니다. 이는 향후 개선이 필요한 영역입니다.

14.7.2 권장 성능 테스트 방법론

14.7.2.1 TPM 연산 성능 테스트

측정 대상:

- TPM 키 생성 시간
- PCR 확장 연산 속도
- 암호화/복호화 처리량
- 증명 프로세스 전체 지연시간

구현 예시:

```
import time

from keylime.tpm import tpm_main


def  benchmark_tpm_operations():

tpm = tpm_main.Tpm()

# 키 생성 성능 측정

start_time = time.time()

for _ in  range(100):

tpm.create_ak()

key_gen_time = (time.time() - start_time) / 100
```

```
# PCR 읽기 성능 측정
start_time = time.time()
for _ in range(1000):
tpm.do_quote()
quote_time = (time.time() - start_time) / 1000
```

14.7.2.2 네트워크 통신 부하 테스트

Verifier 동시 연결 테스트:

```
import asyncio
import aiohttp

async def load_test_verifier(num_agents=100):
"""다중 에이전트 동시 연결 시뮬레이션"""
tasks = []
for i in range(num_agents):
task = asyncio.create_task(simulate_agent_connection(i))
tasks.append(task)
results = await asyncio.gather(*tasks)
return analyze_performance_results(results)
```

14.7.2.3 데이터베이스 성능 테스트

대용량 에이전트 관리 테스트:

```python
def test_database_scalability():
"""수천 개의 에이전트 등록 시 데이터베이스 성능 측정"""
session_manager = SessionManager()
start_time = time.time()
with session_manager.session_context() as session:
for i in range(10000):
agent = VerfierMain(agent_id=f"agent-{i:05d}")
session.add(agent)
session.commit()
insertion_time = time.time() - start_time
# 조회 성능 측정
start_time = time.time()
agents = session.query(VerfierMain).all()
query_time = time.time() - start_time
```

14.7.3 메모리 및 리소스 사용량 모니터링

14.7.3.1 메모리 프로파일링

권장 도구:

- `memory_profiler` : 라인별 메모리 사용량 측정
- `pympler` : 객체 수명 주기 추적
- `tracemalloc` : Python 내장 메모리 추적

```python
from memory_profiler import profile


@profile
def  test_ima_verification_memory():
"""IMA 검증 과정의 메모리 사용량 측정"""
measurements = load_large_measurement_list()
ima.process_measurement_list(None, measurements, runtime_policy)
```

14.7.3.2 CPU 사용률 모니터링

프로파일링 도구:

- `cProfile` : 함수별 실행 시간 측정
- `py-spy` : 실시간 프로파일링
- `perf` : 시스템 레벨 성능 분석

14.8 테스트 데이터 관리

14.8.1 테스트 데이터 구조

`test/data/` 디렉토리는 다양한 테스트 시나리오를 지원하는 데이터를 포함합니다:

인증서 및 키 관리:

- `ca/` : 테스트용 CA 인증서
- `ima_keys/` : IMA 서명 검증용 공개키
- `runtime-policy-*.json` : 다양한 런타임 정책 시나리오

설정 템플릿:

- `templates/` : 버전별 설정 템플릿
- `config/` : 테스트용 설정 파일
- `mappings/` : 설정 매핑 테스트 데이터

14.8.2 테스트 데이터 보안

민감 정보 처리:

- 테스트용 개인키는 실제 운영 환경과 분리
- 테스트 인증서는 자체 서명된 임시 인증서 사용
- 실제 TPM EK 인증서는 테스트에 사용하지 않음

14.9 품질 메트릭 및 모니터링

14.9.1 코드 커버리지 목표

현재 커버리지 설정 (`codecov.yml`):

```yaml
coverage:
status:
project:
default:
informational: true  # 현재는 정보성으로만 사용

flags:
testsuite: {} # 통합 테스트 커버리지
unittests: {} # 단위 테스트 커버리지
```

```
packit-e2e: {} # E2E 테스트 커버리지
```

14.9.2 품질 게이트 기준

자동화된 품질 검사:

1. **코드 스타일**: Pylint, Black, isort 통과
2. **타입 안전성**: MyPy, Pyright 검사 통과
3. **보안 검사**: 정적 분석 도구를 통한 취약점 검사
4. **테스트 통과율**: 모든 단위 테스트 및 통합 테스트 통과

14.10 향후 개선 방향

14.10.1 성능 테스트 프레임워크 구축

권장 구현 사항:

1. **벤치마크 스위트**: TPM 연산, 네트워크 통신, 데이터베이스 성능 측정
2. **부하 테스트 자동화**: 다중 에이전트 시뮬레이션 및 스트레스 테스트
3. **성능 회귀 검출**: CI/CD 파이프라인에 성능 기준선 비교 통합

14.10.2 보안 테스트 강화

추가 보안 검증:

1. **퍼징 테스트**: TPM 명령어 및 네트워크 프로토콜 퍼징
2. **침투 테스트**: 실제 공격 시나리오 시뮬레이션
3. **암호화 강도 검증**: 양자 컴퓨팅 저항성 평가

14.10.3 테스트 자동화 확장

CI/CD 개선 사항:

1. **병렬 테스트 실행**: 테스트 실행 시간 단축
2. **환경별 테스트**: 다양한 OS 및 TPM 버전 지원
3. **성능 모니터링 통합**: 실시간 성능 지표 수집 및 알림

14.11 결론

Keylime 프로젝트는 TPM 기반 원격 증명 시스템의 복잡성에 적합한 포괄적인 테스트 전략을 구축하고 있습니다. 단위 테스트부터 통합 테스트, 보안 테스트까지 다층적인 검증 체계를 통해 시스템의 신뢰성과 보안성을 보장합니다.

특히 SWTPM 에뮬레이터를 활용한 TPM 테스트 환경과 GitHub Actions 기반의 자동화된 CI/CD 파이프라인은 일관되고 재현 가능한 테스트 환경을 제공합니다. 코드 품질 도구들(Pylint, MyPy, Black)의 통합을 통해 코드 품질과 유지보수성을 지속적으로 관리하고 있습니다.

향후 성능 테스트 프레임워크 구축과 보안 테스트 강화를 통해 더욱 견고한 품질 보증 체계를 완성할 수 있을 것입니다. 이러한 테스트 전략은 Keylime이 엔터프라이즈 환경에서 신뢰할 수 있는 TPM 검증 솔루션으로 자리잡는 데 핵심적인 역할을 하고 있습니다.